INSTITUTO PROFESIONAL AIEP

PRO205 – TALLER DE PROGRAMACIÓN

EVALUACIÓN SUMATIVA N.º 3
CASO TECNOBOX

Ciclo de vida del software, principios SOLID y refactorización práctica

1. Introducción

Tecnobox es una distribuidora de insumos tecnológicos que presenta problemas de rigidez, fragilidad y dificultad de mantenimiento en su sistema de inventario. El enunciado plantea analizar las etapas del diseño orientado a objetos, relacionarlas con los modelos Cascada y Ágil, justificar los principios SOLID y demostrar mediante código cómo una refactorización puede mejorar el diseño.

El desarrollo se organiza en dos partes. Primero se presenta el análisis de las etapas y de los principios SOLID. Después se implementa una prueba de concepto con una versión monolítica intencionalmente defectuosa y una solución refactorizada, con pruebas automatizadas.

2. Subtarea 2A: Etapas del diseño orientado a objetos

Las cinco etapas consideradas son análisis, diseño, codificación, pruebas y mantenimiento. El modelo Cascada organiza el trabajo de forma secuencial y procura completar cada fase antes de avanzar; las metodologías Ágiles trabajan en iteraciones, entregan incrementos y utilizan retroalimentación frecuente.

Etapa	Modelo Cascada	Metodologías Ágiles	Aplicación al caso Tecnobox
Análisis	Los requerimientos se levantan y documentan principalmente al inicio. Los cambios tardíos pueden requerir revisar fases anteriores.	Los requerimientos se priorizan y refinan durante el proyecto mediante conversación con usuarios y retroalimentación.	Definir registro de productos, entradas, salidas, consulta de stock y reglas para evitar stock negativo.
Diseño	Se diseña la arquitectura y estructura antes de implementar; los cambios estructurales posteriores pueden ser costosos.	El diseño evoluciona por incrementos, manteniendo una arquitectura suficiente y revisándola a medida que se aprende.	Separar producto, gestión de inventario, persistencia y salida de tickets para evitar acoplamiento.
Codificación	La implementación se realiza después de cerrar análisis y diseño, siguiendo los documentos aprobados.	Se programa en ciclos cortos y se integra código funcional de manera frecuente.	Implementar primero registro y consulta; después entradas, salidas y descuentos.
Pruebas	Las pruebas suelen concentrarse después de la implementación principal, aunque pueden planificarse desde etapas anteriores.	Las pruebas se ejecutan continuamente en cada iteración, para detectar pronto regresiones y errores.	Automatizar pruebas de cálculo de descuentos, stock insuficiente y lectura/escritura de inventario.
Mantenimiento	Las modificaciones se gestionan sobre una solución ya entregada; cambios grandes pueden ser lentos y costosos.	El mantenimiento se integra al trabajo continuo y permite adaptar el producto a necesidades nuevas.	Agregar un nuevo tipo de producto sin modificar el gestor central y conservar pruebas que validen el comportamiento.

2.1. Análisis aplicado

Para Tecnobox, un enfoque Ágil resulta conveniente si las reglas de negocio y las necesidades de los usuarios todavía pueden cambiar, porque permite entregar funcionalidades por etapas y recibir comentarios tempranos. Sin embargo, el modelo Cascada puede ser útil cuando los requerimientos son estables y se necesita una planificación documental muy definida. La elección no significa que uno sea siempre correcto: depende de la estabilidad de los requerimientos, los riesgos y la frecuencia con que se esperan cambios.

En ambos enfoques las cinco etapas siguen siendo necesarias. La diferencia principal está en cómo se ordenan, se repiten y se validan durante el proyecto.

3. Subtarea 2B: Aplicación de los principios SOLID

S — Responsabilidad Única (SRP). Una clase debe tener una responsabilidad principal. Si GestorInventario calcula descuentos, actualiza stock, imprime tickets y escribe archivos, cualquier cambio en impresión o almacenamiento puede afectar a la lógica de inventario. La solución separa los productos, el repositorio de datos y la salida del ticket.

O — Abierto/Cerrado (OCP). El sistema debe permitir agregar comportamientos sin modificar repetidamente código estable. En la versión defectuosa, agregar un tipo de producto obliga a editar una cadena de if/elif. En la refactorizada, cada tipo implementa calcular_descuento() y el gestor utiliza el método polimórfico.

L — Sustitución de Liskov (LSP). Los objetos de una subclase deben poder utilizarse donde se espera la clase base sin romper las reglas del sistema. Los tipos ProductoNormal, ProductoPerecible y LicenciaDigital respetan el contrato de Producto: exponen precio y stock, calculan descuentos válidos y validan las cantidades.

I — Segregación de Interfaces (ISP). Es preferible contar con contratos pequeños y enfocados en lugar de obligar a una clase a implementar operaciones que no necesita. El diseño separa el contrato de persistencia RepositorioInventario del contrato de salida SalidaTicket, de modo que cada componente implementa únicamente la función que le corresponde.

D — Inversión de Dependencias (DIP). La lógica de alto nivel debe depender de abstracciones y no de implementaciones concretas. GestorInventario recibe RepositorioInventario y SalidaTicket mediante su constructor. Por eso puede trabajar con almacenamiento JSON o con un repositorio en memoria para pruebas sin reescribir la lógica.

4. Pregunta complementaria: prueba de concepto y refactorización

4.1. Antes: código monolítico y defectuoso

El archivo gestor_defectuoso.py presenta una versión intencionalmente simple para evidenciar dos problemas solicitados en el enunciado:

- SRP: procesar_venta() calcula el total, arma e imprime el ticket y escribe el archivo en el mismo método.
- OCP: calcular_total() utiliza if/elif por tipo de producto; cada nueva categoría exige modificar el método existente.
- Al mezclar lógica, salida y persistencia, es más difícil probar cada responsabilidad de forma aislada y cambiar una parte sin afectar las demás.

4.2. Después: diseño refactorizado

La versión refactorizada separa las responsabilidades en módulos: productos.py define la abstracción Producto y sus tipos; repositorios.py define el contrato de persistencia y su implementación JSON; salidas.py define la salida del ticket; gestor_inventario.py coordina las operaciones; y main.py inicia la demostración.

- SRP: el producto calcula su descuento; el repositorio guarda/carga datos; la salida muestra el ticket; el gestor coordina la venta.
- OCP y polimorfismo: el gestor invoca producto.calcular_total() sin conocer una lista de tipos concretos.
- DIP: el gestor recibe el repositorio y la salida por constructor, en vez de crearlos internamente.
- Encapsulamiento defensivo: las propiedades del producto se exponen como lectura y las operaciones de stock validan cantidades antes de modificar el estado.

4.3. Demostración de extensibilidad

El proyecto incluye ProductoNormal, ProductoPerecible y LicenciaDigital. Cada clase define su propia política de descuento. La prueba test_nuevo_tipo_producto_se_integra_sin_modificar_gestor crea un tipo adicional derivado de ProductoNormal, sobrescribe calcular_descuento() y lo registra en GestorInventario. El gestor no necesita incorporar una nueva condición por el tipo, porque trabaja contra el contrato de Producto.

La persistencia JSON utiliza un registro automático de subclases concretas de Producto. Al definir una nueva subclase en productos.py, esta se registra al cargarse el módulo y el repositorio puede reconstruirla por su nombre, sin editar la lógica de venta ni el repositorio. La prueba test_nuevo_tipo_producto_se_persiste_sin_editar_repositorio comprueba este comportamiento.

4.4. Métricas de impacto: antes vs. después

Indicador observable	Antes: monolítico	Después: refactorizado	Qué demuestra
Responsabilidades en el flujo de venta	El método procesar_venta() calcula el total, construye/imprime el ticket y escribe en archivo: al menos 3 responsabilidades.	GestorInventario coordina la venta; cálculo, persistencia y salida se delegan a componentes separados.	Mayor cohesión: cada componente tiene una función más definida.
Ramas condicionales por tipo de producto	calcular_total() tiene una cadena if/elif para distinguir normal, perecible y licencia.	El gestor llama a calcular_total() y cada producto implementa calcular_descuento(). No hay if/elif por tipo en la venta.	Mejor extensibilidad: añadir un comportamiento no requiere editar la lógica central de venta.
Separación de responsabilidades	1 módulo/clase concentra inventario, cálculo, impresión y escritura de ticket.	5 módulos principales: productos.py, repositorios.py, salidas.py, gestor_inventario.py y main.py.	Los cambios pueden localizarse en el módulo responsable.
Dependencias de persistencia y salida	El método de venta escribe directamente en un archivo y utiliza print().	El gestor recibe RepositorioInventario y SalidaTicket mediante el constructor.	Menor acoplamiento con el almacenamiento y la presentación.
Pruebas automatizadas	La versión monolítica no incluye pruebas específicas en su archivo.	La versión refactorizada incluye 7 pruebas automatizadas para ventas, descuentos, stock, extensión y persistencia.	Mayor verificabilidad; las pruebas detectan regresiones en comportamientos definidos.
Extensión de un tipo nuevo	Para el nuevo descuento se debe editar el if/elif de calcular_total().	Una subclase implementa calcular_descuento(); el gestor no requiere una nueva condición por tipo.	Cumplimiento práctico del principio Abierto/Cerrado en la lógica de venta.

5. Pruebas y verificación

La suite test_inventario.py incluye pruebas de venta normal, descuento de producto perecible, rechazo de una venta con stock insuficiente, integración de un nuevo tipo de producto y persistencia. También comprueba que un archivo JSON corrupto se comunique mediante una excepción controlada.

En la ejecución preparada para esta entrega, las siete pruebas automatizadas finalizaron correctamente. Los archivos salida_main.txt y resultado_pytest.txt dentro de evidencias contienen la salida de la ejecución.

6. Conclusión

El caso Tecnobox demuestra que los problemas de rigidez, fragilidad y alto acoplamiento pueden reducirse mediante una estructura orientada a objetos con responsabilidades separadas y dependencias dirigidas a abstracciones. La comparación entre Cascada y Ágil permite comprender distintas formas de organizar el ciclo de vida; SOLID aporta criterios para diseñar módulos que puedan probarse y ampliarse. La prueba de concepto muestra la diferencia entre un módulo monolítico y una solución refactorizada, además de verificar comportamientos mediante pruebas automatizadas.

7. Ejecución del proyecto

1. Instalar Python 3.10 o superior.
2. Abrir una terminal en la carpeta codigo.
3. Instalar las pruebas con: python -m pip install pytest
4. Ejecutar el sistema con: python main.py
5. Ejecutar las pruebas con: python -m pytest -v

La entrega final debe incluir el enlace al repositorio GitHub/GitLab del estudiante. El enlace debe crearse desde su propia cuenta y no se inventa en este documento.